

DAY 22: CORRELATION CODING GUIDE

Pearson & Spearman Python Implementations Workbook

Code 1: Pearson Correlation (Pure Manual Implementation)

This implementation calculates the Pearson Correlation Coefficient step-by-step using fundamental loops and basic arithmetic operations without depending on external computational packages.

```

# Step 0: Input Data defining
X = [10, 20, 30]
Y = [3000, 6000, 9000]
N = len(X)

# Step 1: Calculate the Mean (Average) of X and Y
mean_X = sum(X) / N
mean_Y = sum(Y) / N

print(f"Mean of X: {mean_X}")
print(f"Mean of Y: {mean_Y}\n")

# Initialize variables for the formula parts
numerator_sum = 0
sum_deviation_X_sq = 0
sum_deviation_Y_sq = 0

print("--- Step-by-Step Table Calculation ---")
# Step 2: Loop through data to find deviations and products
for i in range(N):
    # Deviation calculation
    dev_X = X[i] - mean_X
    dev_Y = Y[i] - mean_Y

    # Square and product calculations
    product = dev_X * dev_Y
    dev_X_sq = dev_X ** 2
    dev_Y_sq = dev_Y ** 2

    # Running total accumulation ( $\Sigma$ )
    numerator_sum += product
    sum_deviation_X_sq += dev_X_sq
    sum_deviation_Y_sq += dev_Y_sq

    # Print row-by-row simulation just like our PDF table
    print(f"Row {i+1} -> X:{X[i]}, Y:{Y[i]} | (X-Mean): {dev_X}, (Y-Mean): {dev_Y}")

print("\n--- Final Totals ---")
print(f"Numerator ( $\Sigma$  of products): {numerator_sum}")
print(f"Sum of (X-Mean)2: {sum_deviation_X_sq}")
print(f"Sum of (Y-Mean)2: {sum_deviation_Y_sq}")

# Step 3: Apply the values into the Formula
# Denominator = Square Root of [ Sum of (X-Mean)2 * Sum of (Y-Mean)2 ]
denominator = (sum_deviation_X_sq * sum_deviation_Y_sq) ** 0.5
print(f"Denominator: {denominator}")

# Final Pearson Correlation Coefficient 'r'
r = numerator_sum / denominator

```

```
print("\n--- Final Output ---")
print(f"Pearson Correlation Coefficient (r) = {r}")
```

Code 2: Pearson Correlation (Optimized Library Implementation)

This concise syntax utilizes the high-performance scientific package to calculate the linear relationship score along with its statistical two-tailed significance level.

```
from scipy import stats

X = [10, 20, 30]
Y = [3000, 6000, 9000]

print(stats.pearsonr(X, Y))
```

Code 3: Spearman Rank Correlation (Pure Manual Implementation)

This implementation handles ranked variables via static positioning and structural calculations to determine the exact monotonic relationship without programmatic iterations.

```

# Step 0: Data aur unke Ranks (Highest value = Rank 1)
# X = [2, 5, 10] -> Ranks: 2 saal ko Rank 3, 5 saal ko Rank 2, 10 saal ko Rank 1
# Y = [10, 50, 60] -> Ranks: 10 score ko Rank 3, 50 score ko Rank 2, 60 score ko Rank 1

Rx = [3, 2, 1]
Ry = [3, 2, 1]
N = 3

# Step 1: Har row ka direct Difference (d = Rx - Ry) bina kisi loop ke
d1 = Rx[0] - Ry[0] # Row 1: 3 - 3 = 0
d2 = Rx[1] - Ry[1] # Row 2: 2 - 2 = 0
d3 = Rx[2] - Ry[2] # Row 3: 1 - 1 = 0

# Step 2: Har difference ka Square (d²)
d1_sq = d1 ** 2
d2_sq = d2 ** 2
d3_sq = d3 ** 2

# Step 3: Total Sum nikalna (Σd²)
sum_d_square = d1_sq + d2_sq + d3_sq

print(f"Total Sum of d²: {sum_d_square}")

# Step 4: Spearman Formula Apply karna
# Formula: 1 - [ (6 * Σd²) / (N * (N² - 1)) ]
numerator = 6 * sum_d_square
denominator = N * (N**2 - 1)

spearman_rho = 1 - (numerator / denominator)

print(f"Spearman Rank Correlation (ρ) = {spearman_rho}")

```

Code 4: Spearman Rank Correlation (Optimized Library Implementation)

This single-line command runs automated rank transformations and processing loops internally to produce quick outputs for ordinal datasets.

```
from scipy import stats

# Input Data
X = [2, 5, 10]
Y = [10, 50, 60]

# Single-line library calculation
# Yeh function background mein khud ranks nikalta hai, d2 calculate karta hai aur
rho_coefficient, p_value = stats.spearmanr(X, Y)

print(f"Spearman Rank Correlation Coefficient (ρ): {rho_coefficient}")
print(f"p-value: {p_value}")
```

Execution Rule: All question solving structures and numerical logic tables inside the scripts are configured in standardized formats to ensure flawless script execution and clear debugging visualization.